

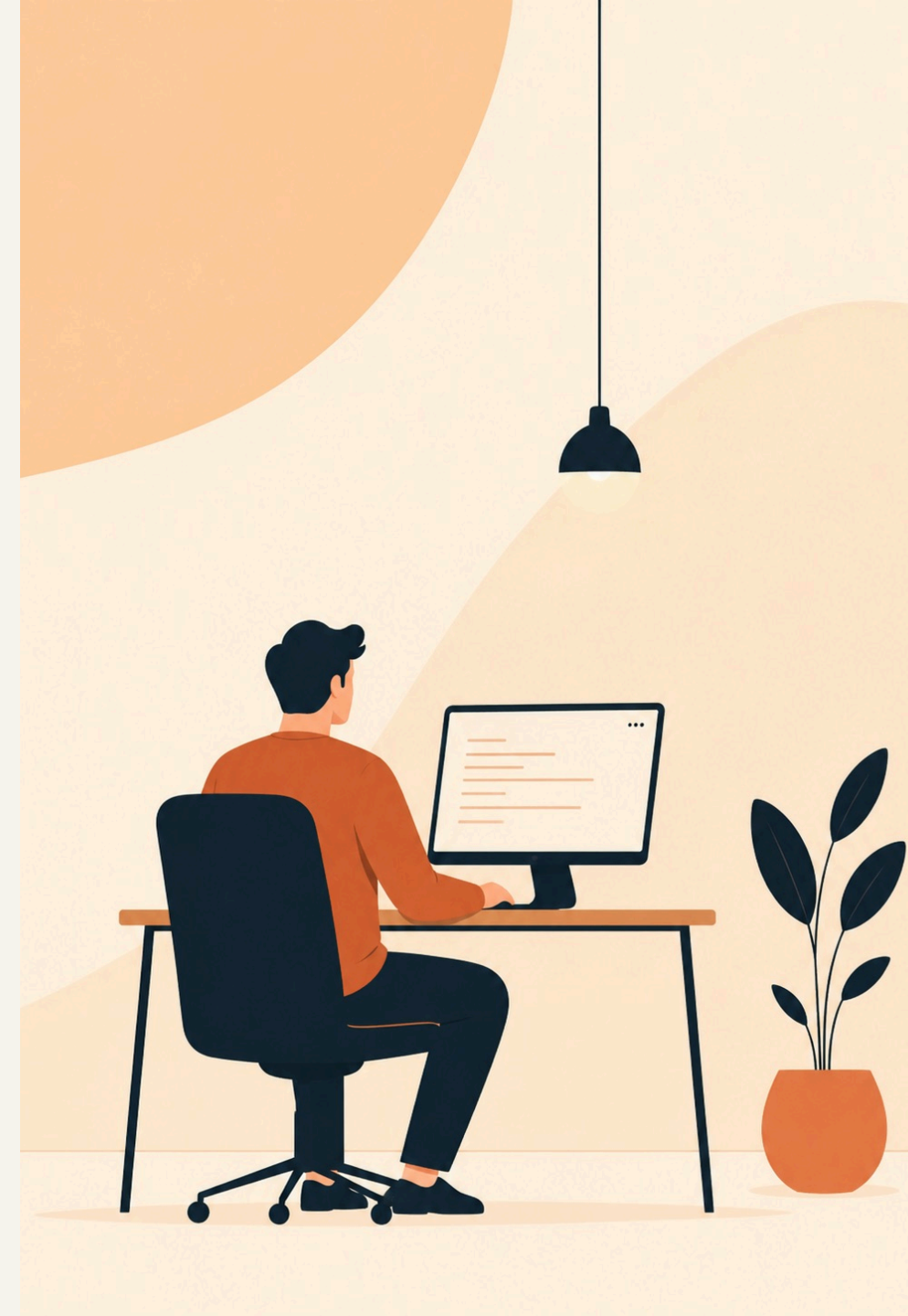
Getting Started with Agentic AI

A practical introduction to navigate the new wave of AI

Fernando Guirao, Uciel Chorostecki, Nicolás Aira

July 22, 2026

UIC Computational RNA Biology Group
barcelona Research Group.



Agenda

1. Introduction: from chat to agents
2. The agent's workspace
3. Getting started with Claude Code
4. Demo: from spec to app
5. Why context matters
6. *Skills*
7. Under the hood: tokens, context, hallucinations
8. How to navigate the hype

1. Introduction

The road to agentic coding

ChatGPT
chat goes
mainstream
Nov 2022

Mar 2023
Cursor
agent inside the IDE

Claude Code
agent goes
standalone
May 2025

Jul 2026
Today

Why this talk?

- The field moves at a brutal pace: new models, benchmarks, and tools every week
- Goal 1: give you a solid foundation to start using it today
- Goal 2: give you judgment to separate signal from noise — **not everything that glitters is gold**

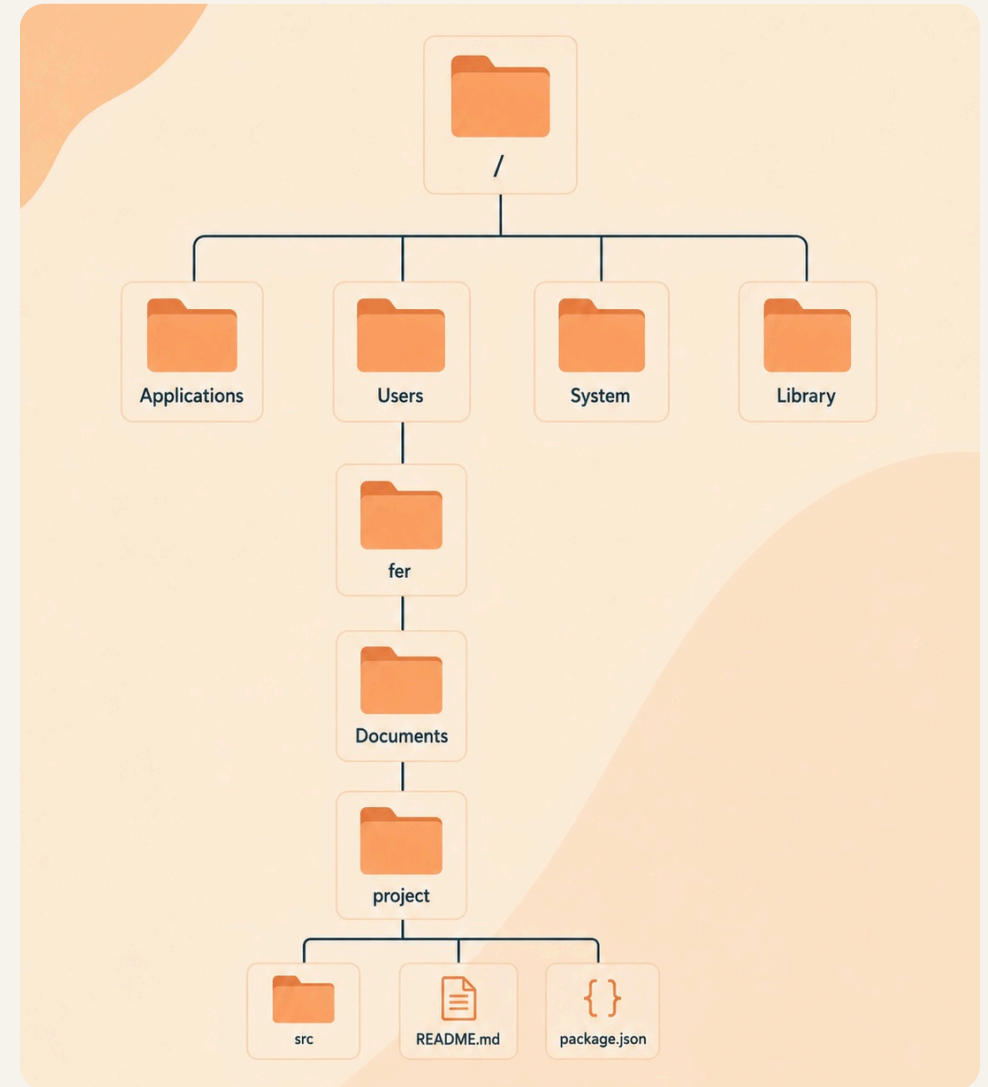
2. The agent's workspace

Where does the agent "live"?

- Tools like Claude Code run **on your own computer**, not just on a website
- They need access to:
 - The **file system** (folders and files)
 - The **terminal** (to run commands)
 - Optionally, the internet

File system basics

- Everything on your computer lives organized in folders (directories) and files
- Absolute path: `/Users/your_username/project/data.csv`
- Relative path: `data.csv` (relative to "where you currently are")
- The agent reads and writes files exactly as you would by hand

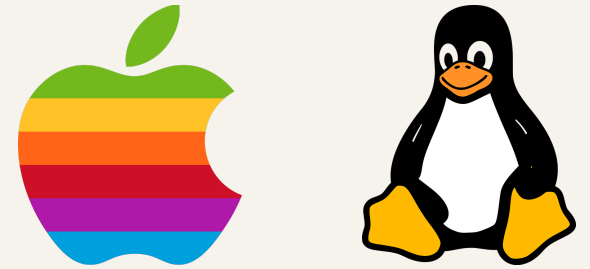


Terminal basics

- The terminal (shell) lets you give the computer instructions by typing text
- Basic commands:
 - `pwd` → where am I?
 - `ls` → what's here?
 - `cd folder` → move into "folder"
- The agent "types" these commands for you
- **Not programming, just running lab protocols:** Think of the terminal as standardizing a recipe. The agent simply runs the steps for you to avoid endless repetitive clicking.

Which operating system to use?

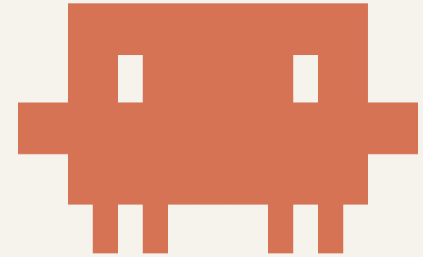
- **Recommended: macOS or Linux**
- **Windows** → **WSL** (Windows Subsystem for Linux)
- The agentic AI tooling ecosystem is built with Unix-like environments in mind



3. Getting started with Claude Code

What is Claude Code?

- A command-line interface (CLI) for working with Claude in an agentic way
- Runs in your terminal, inside your project folder
- Sees your files, can edit them, run commands, install dependencies...
- Can run autonomous tasks in the **background** (e.g., executing a long analysis or refactoring a script while you focus on something else)



Installation

Native installer (recommended, auto-updates):

```
curl -fsSL https://claude.ai/install.sh | bash
```

With Homebrew (macOS/Linux):

```
brew install --cask claude-code
```

Plans and access

- Requires a **Pro, Max, Team, or Enterprise** account (the free claude.ai plan does not include Code)
- For personal use, **Pro (~€20/month)** covers most users
- Alternative: your own API key, pay-as-you-go, no subscription needed

Basic commands

- Start a session: `claude`
- Rename the current session: `/rename`
- Resume a previous session: `claude --resume`
- Exit the session: `/exit`
- Check your usage: `/usage`

Sessions

- A session is **tied to the directory** you launched `claude` from, not to "the project" in the abstract
- Some people always start from the system's root — worth being intentional about where you launch it
- `claude --resume` only lists conversations started in **that same directory**
- Each conversation is stored as a `.jsonl` file (plus a folder) under `~/.claude/projects/<project-folder-path>/`
- You'll also see a `memory/` folder there — more on that later (memory can also live inside your project folder if you want to version it)

4. Demo

From spec to application

- We start from a Markdown file with the **specification** of a small tool
- We hand it to Claude Code and ask it to implement it
- Recommended editor to review the generated code: **VS Code**

What we're going to build

- A small **FASTA sequence analyzer**
- Input: a DNA/RNA sequence in FASTA format
- Output: length, %GC, base composition, reverse complement, approximate Tm
- A simple web page, no backend

(full specification in `fasta-demo-live/spec-demo.md`)

The prompt:

```
Implement the specification in spec-demo.md as a working web page.
```

A more complex case: the ESM-2 Protein Explorer

- Same idea, bigger scope: a full **frontend + backend** webapp instead of a single static page
- Runs a real protein language model (**ESM-2**) on GPU, plus calls to the ESMFold API for 3D structure
- Features: attention-derived contact maps, 3D structure viewer, PCA embedding exploration, in-silico mutation scanning
- Docker Compose setup, GPU detection (CUDA / Apple Silicon MPS), health checks — built the same way, just with more moving parts

(see `webapp-demo/README.md`)

5. Why context matters

"Context" = what the agent knows before it starts

- An LLM has no memory between sessions: it only knows what we tell it in each conversation
- The better the context, the better the decisions the agent makes
- Ways to give context: direct instructions, reference files, project documentation...

'CLAUDE.md' / 'AGENTS.md' — memory management

- CLAUDE.md: a Markdown file Claude Code reads automatically when it starts working in a project
- Contains: project conventions, useful commands, design decisions, things to avoid...
- AGENTS.md: an equivalent, provider-agnostic convention (used by Cursor, Copilot...)
- Claude Code can read both, and can even generate them automatically with `/init`
- Memory normally lives outside the project (`~/.claude/projects/...`) — you can force it into the project folder instead (e.g. a `context/` folder) to version it with git
- CLAUDE.md can index all your context files (memory, lab journaling...) so they're easy to navigate

Context best practices

- Markdown specification documents (like the one used in the demo)
- Keep them short and specific — not a dump of all knowledge in the world
- Context isn't free: it takes up space in the context window (next section)

A real lab example: what goes in `CLAUDE.md` ?

- **Visual standardization:** "Always use our group's specific color palette and clean formatting for RNA expression plots"
- **Tooling defaults:** "Python scripts must use `ViennaRNA` by default for secondary structure folding and analysis"
- **Infrastructure commands:** "Here is the exact command syntax and parameters to submit jobs to our HPC cluster"
- **Data rules:** "Never hardcode absolute file paths; always read sequences using relative paths from the `/data` folder"

6. Skills

What are Skills?

- Reusable packages of instructions for specific, repeatable tasks
- Live in `.claude/skills/`, defined in a `SKILL.md` file
- Invoked either as a command (`/skill-name`) or auto-triggered by the agent when relevant

Why are they useful?

- Only loaded when actually needed → don't take up context in the background
- Let you standardize procedures: "how to deploy", "how to review code", "how to run a type-X analysis"
- Can be shared across projects and across members of the same research group

7. Under the hood

Tokens

- An LLM doesn't process "words", it processes **tokens** (text chunks, ~3-4 characters on average)
- Everything has a token cost: what you write, what the model replies, the files the agent reads
- API pricing and usage limits are measured in tokens

Context window

- The maximum number of tokens a model can "keep in mind" at once
- Includes: the conversation, files the agent has read, `CLAUDE.md`, loaded skills...
- Once it fills up, older content has to be summarized or dropped → detail can be lost
- A very large context window does **not** guarantee equal attention to all of its content

Hallucinations

- An LLM generates statistically plausible text, not necessarily true text
- It can confidently invent functions, citations, or the results of an analysis
- In an agent, this is more serious: it can **take actions** based on a hallucination
- Mitigation: verify, ask for sources, run tests, review the diff before accepting changes

Hallucinations in Biology

- In biology, models can confidently invent **non-existent sequences, fake alignments, or false references**
- In an agent, this is critical: it might run a whole downstream analysis on a hallucinated output
- **The rule:** AI executes and suggests, but scientific validation remains **100% human**

Data confidentiality & science

- **The golden rule:** Never upload unpublished experimental data, sensitive patient sequences, or proprietary lab scripts to public web chats
- **Web Chats vs. Agents/APIs:** CLI agents using direct API keys typically offer much stronger privacy controls and opt-outs from model training
- **When in doubt:** Always check before feeding new sequence datasets or biological data to commercial AI models

8. How to navigate the hype

Some heuristics

- A new model or benchmark every week doesn't mean your workflow should change every week
- Be suspicious of demos you can't reproduce yourself
- Test with your own data and real tasks before believing a headline
- The useful question isn't "is this the newest thing?" but "does this save me time today, on my problem?"
- **Which model to choose?** For agentic coding and bioinformatics scripts, our default recommendation (what Fer uses) is **Claude Sonnet** — the best balance of speed, reasoning, and cost.

Who to follow (great tech creators in Spanish)

- **midudev** — web & coding best practices
- **BettaTech** — software engineering & tech concepts
- **La Inteligencia Artificial** — AI updates & model overviews

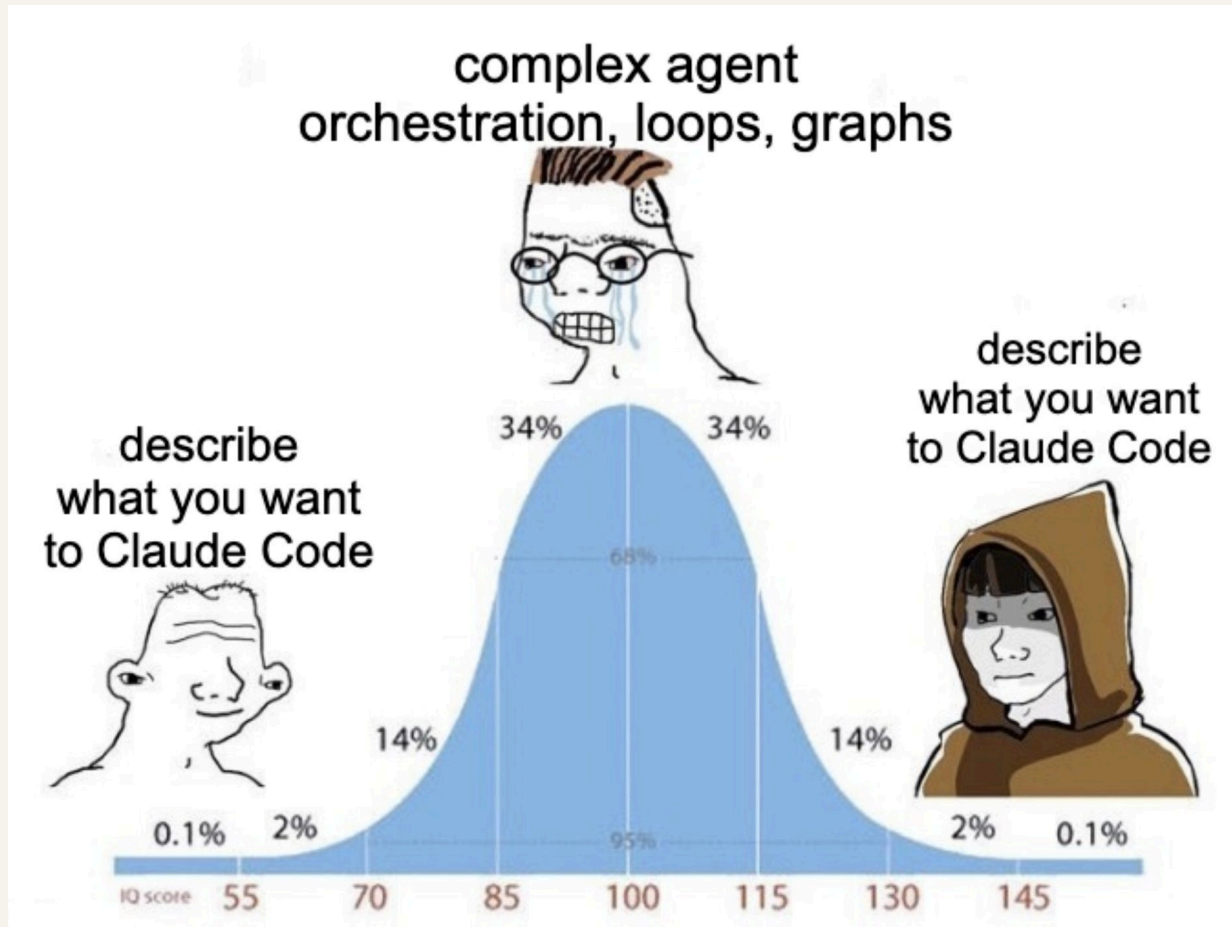
Hands-on: RNA-seq analysis with ClawBio

- Try it yourself: an end-to-end **bulk RNA-seq** workflow — quantification, QC, and differential expression — driven entirely by natural-language prompts to Claude Code
- Uses the **ClawBio** plugin (Salmon quantification, MultiQC) plus a custom `differentialabundance` skill wrapping the nf-core/differentialabundance Nextflow pipeline
- Shows Claude Code orchestrating real bioinformatics pipelines (Salmon, MultiQC, nf-core), not just writing code

(full instructions in `rnaseq_demo/README.md`)

Relax.

You probably don't need a 12-agent orchestration graph. Just say what you want.



Open discussion & lab pro-tips

Let's hear from the team

- **From our AI experts:** What are your top daily tricks for terminal agents, prompts, and token optimization?
- **From our biomed undergrads:** What repetitive data or lab task would you like an agent to automate for you tomorrow?
- **Q&A:** How can we start integrating this into our current RNA workflows?

Thank you!

Questions

Fernando Guirao — fernando.guirao@estudiantat.upc.edu

Uciel Chorostecki — upchorostecki@uic.es

Nicolás Aira — naira@uic.es